# Inheritance in Python

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

# What is Inheritance?

**Definition**: reuse of attributes and methods from a base class to a derived class.

**Syntax:**

```
class BaseClass:
    def method_base(self) -> None:
      pass # base class method

class DerivedClass(BaseClass):
    def method_derived(self) -> None:
        pass # derived class method
```

| Component | Description |
|---|---|
| `BaseClass` | The parent (superclass) that defines shared behavior |
| `DerivedClass` | The child (subclass) that inherits from `BaseClass` |
| `:` | Declares the beginning of the class definition |
| `()` | Used to specify the parent class being inherited |

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

# Inheritance – Basic Example

Base class

```
class Animal:
    def speak(self) -> None:
      print("The animal makes a sound")
```

Inherited class

```
class Dog(Animal):
    def bark(self) -> None:
      print("Woof!")

Fido:Dog = Dog()
fido.speak() # Output: "The animal makes a sound"
fido.bark() # Output: "Woof!"
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

# Inheritance – super().__init__()/1

**Why use super()?**

- Calls the parent class's `__init__()` method.
- Ensures proper initialization of inherited attributes.
- Especially useful in multi-level or multiple inheritance.

**Best Practices**

- Always use `super().__init__()` when extending `__init__()` in child classes.
- Avoid repeating parent class initialization manually.

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

# Inheritance – super().__init__()/2

```python
# Without super()
class Animal:
  def __init__(self, name:str) -> None:
    self.name:str = name
    print("Animal initialized")

class Dog(Animal):
  def __init__(self, name:str) -> None:
    self.name:str = name
    print("Dog initialized")

fido:Dog = Dog("Rudy")
# Output: Dog initialized
```

```python
# With super()
class Animal:
  def __init__(self, name:str) -> None:
    self.name:str = name
    print("Animal initialized")

class Dog(Animal):
  def __init__(self, name:str) -> None:
    super().__init__(name)
    print("Dog initialized")

fido:Dog = Dog("Rudy")
# Output: Animal initialized
# Dog initialized
```

Parent's class __init__()

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

# Method Overriding

**Definition**

- **Method overriding** occurs when a **subclass defines a method** with the **same name** and **parameters** as a method in its **superclass**.
- The subclass version **replaces** the one inherited from the parent class.

**Purpose**

- Customize or extend the behavior of a method inherited from the parent class.
- Achieve **polymorphism**: same interface, different behaviors.

its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

# Method Overriding – Basic Example

```
class Animal:
    def speak(self) -> None:
      print("The animal makes a sound")


class Cat(Animal):
    def speak(self) -> None:
      print("Meow!")

kitty:Cat = Cat()
kitty.speak() # Output: "Meow!"
```

Method override

**Key Characteristics**

- Method names and signatures must match.
- Only methods (not attributes) can be overridden.

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

# Protected Attributes and Methods/1

**What are Protected Members?**

- A **protected attribute or method** is **intended** to be accessed only **within the class** and its **subclasses**.
- Defined using a **single underscore** prefix: _attribute or _method().

❗ **Important Note**

- Python doesn't **enforce** access restrictions like Java or C++; it uses **conventions**.
- Developers are expected to **respect** the underscore as a **"don't touch unless you're a subclass"** warning.

ISTITUTO TECNOLOGICO SUPERIORE ACADEMY - INFORMATION AND COMMUNICATIONS TECHNOLOGY
its ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY

# Protected Attributes and Methods/2

```
class Animal:
  def __init__(self) -> None:
    self._type:str = "Mammal" # Protected attribute

  def _sound(self) -> None: # Protected method
    print("Generic animal sound")

class Dog(Animal):
  def describe(self) -> None:
    print(f"I am a {self._type}") # Accessing protected attribute
    self._sound() # Calling protected method

fido:Dog = Dog()
fido.describe() # Output: "I am a Mammal"
                # Generic animal sound
```

ICTAcademy
INFORMATION AND COMMUNICATIONS TECHNOLOGY